

Safety-constrained Reinforcement Learning with Interaction-aware for Decision-making of Autonomous Driving

Di Zhang[†], Haonan Luo^{‡*}, Honglin Dong[†], Jianfeng Lu[†]

[†]School of Computer Science and Engineering, Nanjing University of Science and Technology, China

[‡]School of Computer and Artificial Intelligence, Southwest Jiaotong University, China

Email: zdd@njust.edu.cn, lhn@swjtu.edu.cn, dshuh@163.com, lujf@njust.edu.cn

Abstract—Reinforcement learning(RL) has made significant advancements in autonomous driving(AD). However, the stochastic nature of dynamic traffic scenario and the diversity of road type make it challenging for autonomous vehicles to make safe and efficient decisions. To tackle these problems, this paper proposes a novel RL framework that incorporates the motion prediction model to enhance the agent's decision-making capability. We first utilize Transformer to model driving scenarios and capture interaction-aware relationships between the ego vehicle and scenarios, then design a safety-constraint and integrate it into the Proximal Policy Optimization (PPO) algorithm so as to guarantee the safety and feasibility of the policy. To improve data efficiency and filter noisy samples, we construct a dual network to communicate and guide each other. Experimental results show that compared with popular RL algorithms, our method demonstrates superior performance in success rate, completion time, safety, and data efficiency.

Index Terms—Autonomous Driving, reinforcement learning, decision-making

I. INTRODUCTION

As research on reinforcement learning delves deeper, it has proven successful in numerous fields. Researchers wish to apply it to more complex tasks, particularly in the field of autonomous driving, which has received widespread attention and research [1], [2]. The decision-making module, as the key component of autonomous driving, must make the safe and efficient decisions. However, it still faces great challenges due to the stochastic nature of dynamic traffic scenario and the diversity of road type. Reinforcement learning allows the vehicle to learn from its interactions with the driving environment and iteratively optimize the driving strategy [3]–[5]. Combining the powerful representational capabilities of deep learning to model driving scenarios offers more possibilities for decision-making in autonomous driving.

Nonetheless, RL-based autonomous driving techniques still face some challenges. The first issue is the problem of scene representation. Effective scene representation is crucial for the decision-making process of autonomous vehicles, as it is essential for the agent's comprehension of the driving scene. When facing with complex traffic information from multiple heterogeneous sources such as vehicle dynamics observation information, inter-vehicle interaction information,

and road information, it becomes vital to identify appropriate methods for characterizing the environment to extract relevant information. Furthermore, the issue of safety is one of the main concerns in translating simulation results into reality. It is important to prevent the vehicle from entering an infeasible state for maintaining the safety of autonomous driving. The infeasible state may cause the model to make high-risk or unsafe decisions, such as sharp turns, sudden brakes, etc., thereby increasing the risk of accidents. The infeasible state may also lead to instability and irrationality of the strategy, causing the vehicle to deviate from the normal driving route. Beyond that, we should also ensure that the vehicle can make optimal choices in infeasible states. Therefore, reinforcement learning must prioritize safety when dealing with autonomous driving scenarios. Finally, the challenge of data efficiency is also an issue. Reinforcement learning necessitates a substantial amount of interaction data for training, consequently prolonging the training time. Meanwhile, the interaction data contains noisy samples, which can affect the accuracy of the model and destabilize the performance of the strategy. This problem can become even more acute when dealing with complex tasks like autonomous driving. Therefore, it is necessary to explore methods to improve the data efficiency of reinforcement learning. Addressing these issues will be crucial for advancing RL-based autonomous driving technologies.

In this paper, we present a novel reinforcement learning framework to address these issues for autonomous driving decision-making. The main contributions are summarized as follows:

- We propose a reinforcement learning framework that incorporates motion prediction models to enhance decision-making effectively. The driving scenarios are encoded using Transformer to help the agent understand the driving environment effectively.
- A safety-constraint is developed to ensure the safety and viability of reinforcement learning for making decisions.
- Constructing a dual network to communicate and guide with each other to filter noisy samples and improve data efficiency.
- We evaluate our framework in several challenging driving scenarios and assessed the key components of the framework as well. Compared to the baseline approach, our method

* Corresponding author

demonstrates superior performance and data efficiency.

II. RELATED WORK

Reinforcement learning can learn optimal policies by interacting with the environment without preset complex rules [6], and is therefore widely used in autonomous driving. In addition to reinforcement learning, imitation learning(IL) [7] has also garnered significant attention in the field of autonomous driving. However, IL requires a large amount of expert data to guide the agent's learning strategy, which has a much higher cost compared to RL, and there is also a problem of distributional bias in deployment [8]. In contrast, reinforcement learning demonstrates its strengths. Researchers have studied many individual tasks such as merging [9], [10], car following [11], and lane keeping [12], [13] for autonomous driving. Due to the diversity of driving scenario, only single strategy is not enough for autonomous driving. The idea of hierarchical policy is presented in [14]–[16]. Guo et al. [15] proposes a strategy-switching method based on hierarchical reinforcement learning to switch between strategies of different driving styles. Sun et al. [14] designs auxiliary tasks to learn the characteristics of various scenarios, enabling agents to adopt distinct strategies for different scenarios. Wang et al. [16] transforms hierarchical behavior and motion planning by utilizing a classical sample-based motion planner, converting optimal costs into rewards for advanced behavioral learning.

However, policy switching must be based on a thorough understanding of the environment. With the development of Transformer and Graph Neural Network (GNN), researchers use representation learning to help agents understand the scenario [17], [18]. GNN is used to encode the various object types in the driving scenario [19]–[21]. Cai et al. [19] proposes DG-GAT for scalable autonomous driving and utilizes a graph attention-based network to implicitly model interactions, process heterogeneous traffic information in various driving scenarios, design reward functions, and train the network using dueling double deep Q-learning (D3QN). Shi et al. [20] predicts the multimodal movements of multiple agents simultaneously and facilitates future behavioral interactions among multiple agents. Liu et al. [22] constructs a multistage transformer (MST) encoder to simulate the interaction perception between the ego vehicle and the scenario. Huang et al. [23] incorporates the future plans of the ego vehicle into the decoding process to predict the future trajectories of surrounding vehicles and select the optimal candidate path for execution.

Autonomous driving has high requirement for safety, so unsafe maneuvers need to be avoided when making decisions. And safety-constrained reinforcement learning [24]–[26] can be a good solution to this problem. Safety-constraint reinforcement learning can be divided into two categories: Lagrangian approach and Lyapunov approach. Ma et al. [24] adopts an additional neural network to first approximate the statewise Lagrange multiplier and then construct a novel statewise Lagrange function. Zhang et al. [25] proposes a first-order method named Exact Penalty Optimization (EPO) and demonstrate its capability in safe AD.

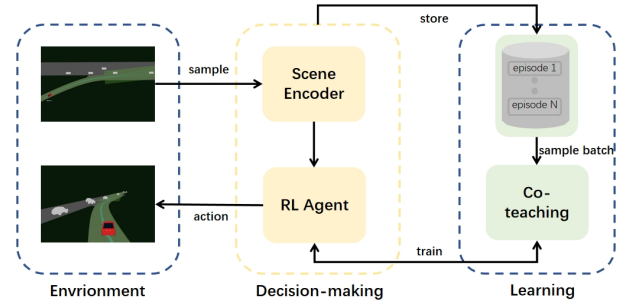


Fig. 1. Overview of our proposed framework. The agent first interacts with the environment. Scene encoder encodes the scene inputs to RL for decision-making, and stores the samples in the replay buffer. The agent accepts the action and executes it in the next step. Sample batch data from the replay buffer to train the agent.

III. METHOD

The proposed RL framework aims to generate safe and efficient driving strategies in complex scenarios, as illustrated in Fig. 1. Our framework is divided into three main parts: the first part is the environment interaction. The vehicle interacts with the driving scenario and transmits perceptual information to the scene encoder. And the second part is the decision-making component, which includes the scene encoder and RL agent. Scene encoder utilizes Transformer to extract interaction-aware features. Using the features as inputs to Safe-RL to make decisions. Details of the decision-making module are described in Fig. 2. The third part involves training, which utilizes co-teaching to establish a dual network for training the agent.

A. Decision-making

The Decision-making module consists of two parts, and Fig. 2 shows the details. Scene information is processed through various network structures to establish interaction-aware relationships between the ego vehicle and the scene. The potential feature vectors are then fed into the actor network of RL in order to decode and generate appropriate actions.

Scene encoder. In order to enable autonomous vehicles to better understand the environment, we selected the historical trajectories of the ego vehicle ($O_{-t:0}^e$), neighboring vehicles ($O_{-t:0}^i$), and candidate routes (M_t^c) as inputs for scene encoder at every timestep t . The number of neighboring vehicles, historical trajectories, and candidate routes are set to a fixed number, and any missing parts are padded with zeros. We encode the vehicles' states and candidate routes at every timestep using an agent-encoder and a map-encoder, respectively, which are different multilayer perceptron(MLP) networks. Then, a self-attention layer [27] is used to encode each agent's historical trajectories and obtain the temporal relationship, furthermore, we treat each temporal relationship as a potential representation, utilizing two self-attention layers to establish interactions between agents. The agent's interaction features are used as query, the map-encoded features as key and value, and two cross-attention layers are adopted to capture the relationship between the agent and the candidate routes. Finally,

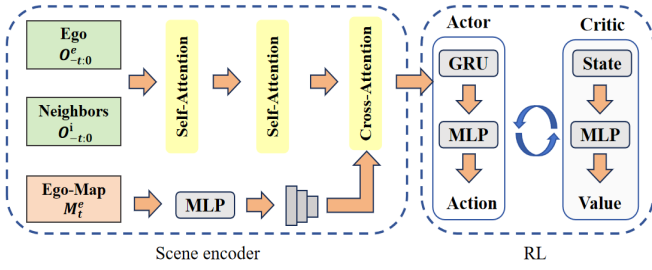


Fig. 2. Overview of decision-making module. The historical trajectories of each vehicle ($O_{t:0}$) are first processed using self-attention to establish temporal relationships. Subsequently, the relationships between the ego vehicle and surrounding vehicles are determined through two self-attention layers. Finally, the relationship between the ego vehicle and candidate routes (M_t) is established using two cross-attention layers, which are used to decode the output action ($\delta x, \delta y, \delta \theta$) through the GRU.

we concatenate the ego's features, ego-neighbors features, and ego-map features as hidden features (h_t) and pass to the actor network.

Actor-Critic. The actor network of PPO is used to decode and make decision. Actor network accepts the hidden features h_t of the scene encoder as input. We utilize the Gated Recurrent Unit (GRU) [28] to decode the hidden feature vectors h_t , and the decoded state is used as the relative deviation of the current pose of ego vehicle. Unlike the actor network, we only use MLP in the critic network. Since the interaction-aware module is only used to make decisions, instead of using the hidden feature vectors h_t as input to the critic network, we directly feed it the pre-processed data.

B. Safety-Constraint for Proximal Policy Optimization

PPO is a RL algorithm based on policy gradient. It can achieve stable and efficient training results by limiting the magnitude of policy updates, and allow for multiple small batches of updates [29]. The main objective of PPO is as the following:

$$L^{clip}(\theta) = \hat{\mathbb{E}}_t[\min(r_t(\theta)\hat{A}_t, \text{clip}(r_t(\theta), 1 - \varepsilon, 1 + \varepsilon)\hat{A}_t)] \quad (1)$$

where π_θ is a stochastic policy, $r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{old}}(a_t|s_t)}$ denotes the probability ratio of the old and new policy, and $\hat{A}_t$ is the advantage function, ε is a hyperparameter used to control the clipping magnitude [29]. PPO modifies the surrogate objective by clipping the probability ratio.

A learned state-value function $V(s)$ is used to compute variance-reduced advantage-function estimators. The value function loss is defined as follows:

$$L^{vf}(\theta) = \frac{1}{2} \mathbb{E}[(V_\theta(s) - V_{target})^2] \quad (2)$$

PPO uses a neural network architecture that shares parameters between the policy and value function. Thus, we use a joint loss function to optimize the objective as follow:

$$L(\theta) = L^{clip}(\theta) - c_1 L^{vf}(\theta) + c_2 S[\pi_\theta] \quad (3)$$

where c_1 and c_2 are hyperparameters used to balance the weights of different loss functions, $S[\pi_\theta]$ is an entropy bonus, aiming to enhance the explorability of the policy [29].

Safety-constraint. We use a Lagrangian-based approach to solve the safety-constraint problem. The constrained optimization problem is transformed into an unconstrained optimization problem using a Lagrange multiplier. We set cost values for all infeasible states. For example, if a vehicle pulls out of a lane while traveling, we would set the cost return value to 1 to indicate that the state is infeasible. Given a policy π_θ , we define that a state is safe if its expected return of cost V_θ^c satisfies the inequality [24]:

$$V_\theta^c(s) \leq d \quad (4)$$

where d is cost threshold. Same as the value loss function, the cost function loss is as follows:

$$L^c(\theta) = E[(V_\theta^c(s) - V_{target}^c)^2] \quad (5)$$

Thus, constrained optimization becomes unconstrained optimization. The optimization objective of PPO with safety-constraint transforms:

$$L(\theta) = L^{clip}(\theta) - c_1 L^{vf}(\theta) + c_2 S[\pi_\theta] - L^c(\theta) - L^{lam}(\theta) \quad (6)$$

where $L^{lam} = \mathbb{E}[\lambda(s)(V_\theta^c(s) - d)]$, and $\lambda(s)$ is the Lagrange multiplier. When updating the policy network, L^{lam} just participate in computation not back propagate, which is a separate network. After policy optimization, it will be optimized by its own loss function.

C. Co-teaching

To improve data efficiency, we introduce co-teaching [30] to train two separate networks ω_1 and ω_2 simultaneously. They have the identical network structure, but different parameters. They all have their own actor network and critic network. In each mini-batch of data, we forward propagate the two networks separately. Each network selects the instances with small loss values as valid samples (Y_1 is the valid samples of ω_1 and Y_2 is the valid samples of ω_2) and passes them to the other network for training. Therefore, their loss functions are:

$$L(\omega_1) = f(Y_2; \omega_1) \quad (7)$$

$$L(\omega_2) = f(Y_1; \omega_2) \quad (8)$$

We set a forgetting rate ($R(t)$) that increases with the number of training steps:

$$R(t) = \begin{cases} 0, & S_{current} < S_{threshold} \\ \frac{S_{current} - S_{threshold}}{S_{total}} * \tau, & \text{else} \end{cases} \quad (9)$$

where τ is hyperparameter used to control the forgetting rate and $\tau \in [0, 1]$, $S_{current}$ is the current step count, $S_{threshold}$ is the step threshold, and S_{total} is the total number of steps. The number of samples passed is $(1 - R(t)) * |Y|$. In the early stages of training, our goal is to encourage the agent to explore the environment extensively to prevent model overfitting and improve its robustness. Therefore, we set $R(t)$ to zero. Consider all samples as valid instances. As the number of training rounds increased, $R(t)$ also increased. We achieved fine-tuning of the model by filtering out the samples with larger loss and not using them for backpropagation.

TABLE I
PARAMETERS USED IN THE EXPERIMENT

Notation	Meaning	Value
η	Learning rate	0.0003
d	Cost threshold	0.1
γ	Discount factor for reward	0.99
γ_c	Discount factor for cost	0.99
τ	Control factor for forgetting rate	20%
$S_{threshold}$	Step threshold	100000

TABLE II
THE DETAILS OF SCENARIO INFORMATION

Description	Value
Speed limit(m/s)	10
Lane width(m)	3.2
Number of neighboring vehicles	5
Max number of lanes	3
Vehicle size(m)	[3.68, 1.47, 1.4]

IV. EXPERIMENT

A. Experiment setup

All the experiments are conducted on a NVIDIA TITAN XP GPU with Adam optimizer. We train the model for 700,000 steps in each scenario, and update network parameters for 80 rounds after every 3000 steps. The relevant parameters of the experiment are shown in table I:

Driving scenarios. We train and test our method in three highly interactive scenarios in the SMARTS simulator [31], which are left turn, merge, and overtake as shown in Fig. 3. The left turn scenario requires the vehicle to pass through the intersection in both lanes and complete a left turn at the intersection. In the merge scenario, vehicles need to merge from a one-lane side road into a three-lane main road. In the overtake scenario, the vehicle needs pass the car in front of it on a one-way three-lane road and reach the target. Table II shows the details about the scenarios.

State space. At each timestep t , we select the 11 steps(where one step is 0.1 second) historical trajectories of ego vehicle and five neighboring vehicles, which include position, angle, and speed. For ego vehicle, we choose three nearby routes with fifty waypoints as map. Every waypoint is one meter apart. Waypoints contain position, angle, and speed limit. We convert these values

to relative values for the current pose of the ego vehicle. These data will be used as input for scene encoder.

Action space. We use continuous action space, it is the vehicle's next pose in terms of delta x coordinate, delta y coordinate, and delta heading, to be reached in 0.1 second. And it is the relative value of the current pose. The maximum value of the action space was adjusted to take into

TABLE III
ACTION SPACE

Name	Max	Min
δx	1	0
δy	0.2	0
$\delta \theta$	0.157	-0.157
δt	0.1	0.1

account the restrictive nature of driving scenarios(speed limits, backtracking, etc.) and driving habits. As shown in table III.

Reward and cost. The results of strategies trained using the simulator's default reward function are poor, so we designed a novel reward function instead of using the original reward function. It consists of seven parts, which are: distance traveled reward, events reward, goal extra reward, distance to goal reward, distance to center reward, safe distance reward, and speed reward. We have carefully designed every part to perform well in our experiments. The total reward can be expressed as follows:

$$r = r_d + r_e + r_{ge} + r_{d2g} + r_{d2c} + r_{safe} + r_{speed} \quad (10)$$

Three states are defined as infeasible states, which are collision, off-road, and off-route. Hence, the cost for these three states is set to 1, and 0 for other states.

B. Results of evaluation

We evaluated our method, components, and the baseline methods in three different interaction scenarios. Each scenario was randomly iterated for 40 rounds, with success rate and average completion time as our metrics.

Compared with baseline methods. Table IV shows the results of our method compared to the three baseline methods. In terms of both success rate and average completion time, our method far outperforms the three baseline methods, achieving true safety and efficiency. In all three scenarios, TD3 performs the worst, indicating that it is not suitable for autonomous driving tasks. PPO performs the best among the three baseline methods, benefiting from its stability, but it's still inferior to our method. Of the three scenarios, the merge scenario has the lowest success rate. It has more complex interactions compared to the other two scenarios, but our method still can achieve the success rate of 94%.

Ablation study. To evaluate the various components of our method, we conducted ablation experiments in three driving scenarios. Table V shows the results. Without the interaction-aware module, the model performs poorly in the ablation experiments. The success rate of the best performing is in overtake scenario, it is 78%, and it is even only 65% in the merge scenario. Interaction-aware is crucial for the agent to correctly understand of the driving environment and is an essential element for making decisions. By accurately

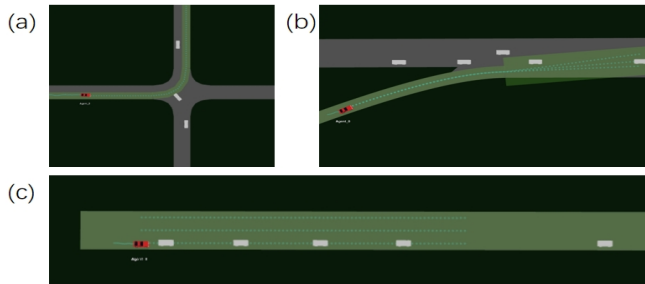


Fig. 3. Three driving scenario in SMARTS: (a) left turn, (b) merge, (c) overtake.

TABLE IV
TEST RESULTS OF DIFFERENT METHODS IN THREE SCENARIOS

Method	Left turn		Merge		Overtake		Overall
	Success	Time	Success	Time	Success	Time	Success
TD3	12%	28.4s	7%	31.8s	37%	33.6s	18.7%
SAC	35%	20.6s	33%	26.1s	48%	26.1s	38.7%
PPO	54%	23.8s	42%	25.4s	58%	28.3s	51.3%
Ours	97%	17.5s	94%	23.8s	100%	24.4s	97%

TABLE V
TEST RESULTS OF ABLATED METHODS IN THREE SCENARIOS

Method	Left turn		Merge		Overtake		Overall
	Success	Time	Success	Time	Success	Time	Success
w/o interaction-aware	74%	19.9s	65%	27.3s	78%	29.1s	72.3%
w/o safety-constraint	86%	17.1s	76%	22.6s	90%	26.2s	84%
w/o co-teaching	92%	18.2s	92%	24.5s	100%	25.4s	94.7%
Ours	97%	17.5s	94%	23.8s	100%	24.4s	97%

modeling the information within the scene, the agent learns that relationships with other influencing elements, which is an integral part of the autonomous driving task. In the absence of the safety-constraint module, the agent does not fully consider the feasibility of states and the optimal policy in infeasible states. As a result, it behaves more aggressively, particularly in left turn and merge scenarios, although the completion time is shorter than that of our method, the success rate is lower compared to our method. In autonomous driving, safety is undoubtedly the primary concern. Co-teaching provides a slight enhancement to the model. The most noticeable improvement is a 5% increase in the left turn scenario. In relatively simple driving scenarios, the interaction-aware module can perfectly capture scene information, in which case the influence of noise samples on the model will become weak. However, there is still an improvement in the completion time, indicating that co-teaching will still focus more on the learning of valid samples, which attributes to its ability to filter out noisy samples and thus facilitate model fine-tuning.

C. Comparative analysis

We trained our methods, baseline methods, and ablated methods in three different scenarios. As shown in Fig. 4 and 5, which are the training curves of our method are compared with current popular RL methods and ablated methods. The metrics that measure the performance of the model are the reward per episode and the success rate.

Fig. 4(a) illustrates that our method achieves higher rewards, converges faster, and is more stable than other RL algorithms at the same training step length. TD3 was not stable enough during training compared to other methods and still exhibited fluctuations late in the training process. At approximately 200,000 steps, our method is nearing convergence, whereas the other RL methods are still in the learning phase. As shown in Fig. 4(b), our method has a significantly higher success rate. Even though the reward after PPO convergence is not very different from the reward of our method, there is still a significant disparity in the success rate. This is attributed to the fact that the agent does not learn the correct strategy.

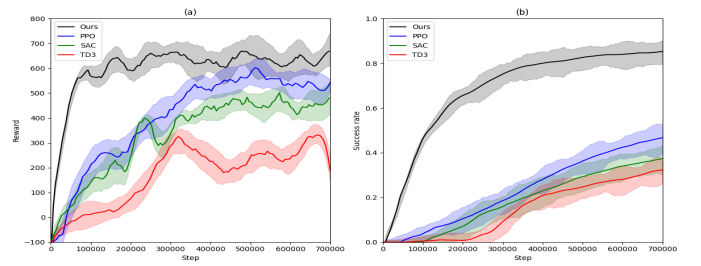


Fig. 4. Training curves for our method compared to the baseline method. (a) represents the reward and (b) indicates the success rate. The black curve is **our proposed method**, the others are blue(PPO), green(SAC), and red(TD3).

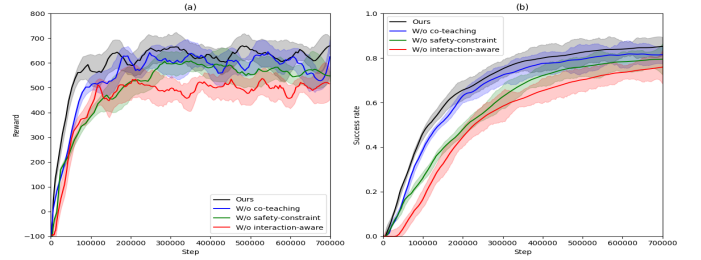


Fig. 5. Training curves for ablation methods. The black curve is **our proposed method**, the others are blue(**w/o co-teaching**), green(**w/o safety-constraint**), and red(**w/o interaction-aware**).

We performed ablation experiments on our proposed framework, and the results are shown in Fig. 5. In the absence of the interaction-aware module, rewards and success rates are minimal. The interaction-aware module has a greater impact on our framework compared to other components. The safety-constraint and co-teaching also contribute to the improvement in the performance of our model. The method without co-teaching module is the closest to our method in terms of either reward or success rate. Although the final results are close, our method learns faster. As the number of training steps increases, the forgetting rate also increases. We only use samples with smaller loss for backpropagation. Consequently, the training curve of our method becomes very smooth in the

later stages of training, it is just a fine-tuning process. But at around 600,000 steps, the method without co-teaching module oscillates.

V. CONCLUSION

We propose a novel reinforcement learning framework for making safe and efficient decisions for autonomous driving. Our approach first models the driving scenario by using Transformer, which is decoded in the actor network in PPO to make decisions. We develop a safety-constraint in RL to guarantee the feasibility of the driving state and employ an additional neural network to estimate the Lagrange multiplier. Finally, a dual network is constructed using co-teaching to enhance data efficiency and fine-tune the model. We evaluate our method in three interactive driving scenarios. The experimental results demonstrate that our method has advantages in terms of success rate, reward, driving efficiency, and data efficiency.

REFERENCES

- [1] Bangalore Ravi Kiran, Ibrahim Sobh, Victor Talpaert, Patrick Mannion, Ahmad A. Al Sallab, Senthil Yogamani, and Patrick Pérez. Deep reinforcement learning for autonomous driving: A survey. *Cornell University - arXiv, IEEE Transactions on Intelligent Transportation Systems*, Feb 2020.
- [2] Zeyu Zhu and Huijing Zhao. A survey of deep rl and il for autonomous driving policy learning. *IEEE Transactions on Intelligent Transportation Systems*, page 14043–14065, Sep 2022.
- [3] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Alex Graves, Ioannis Antonoglou, Daan Wierstra, and Martin Riedmiller. Playing atari with deep reinforcement learning. *arXiv: Learning, arXiv: Learning*, Dec 2013.
- [4] Tuomas Haarnoja, Aurick Zhou, Pieter Abbeel, and Sergey Levine. Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. *arXiv: Learning, arXiv: Learning*, Jan 2018.
- [5] Scott Fujimoto, Herkevan Hoof, and David Meger. Addressing function approximation error in actor-critic methods. *arXiv: Artificial Intelligence, arXiv: Artificial Intelligence*, Feb 2018.
- [6] Jianyu Chen, Bodi Yuan, and Masayoshi Tomizuka. Model-free deep reinforcement learning for urban autonomous driving. In *2019 IEEE Intelligent Transportation Systems Conference (ITSC)*, Oct 2019.
- [7] Jianyu Chen, Bodi Yuan, and Masayoshi Tomizuka. Deep imitation learning for autonomous driving in generic urban scenarios with enhanced safety. *Cornell University - arXiv, Cornell University - arXiv*, Mar 2019.
- [8] Scott Emmons, Benjamin Eysenbach, Ilya Kostrikov, and Sergey Levine. Rvs: What is essential for offline rl via supervised learning?
- [9] Pin Wang and Ching-Yao Chan. Autonomous ramp merge maneuver based on reinforcement learning with continuous action space. *arXiv: Artificial Intelligence, arXiv: Artificial Intelligence*, Mar 2018.
- [10] Maxime Bouton, Alireza Nakhaei, Kikuo Fujimura, and Mykel J. Kochenderfer. Cooperation-aware reinforcement learning for merging in dense traffic. *arXiv: Robotics, arXiv: Robotics*, Jun 2019.
- [11] Meixin Zhu, Xuesong Wang, and Yinhai Wang. Human-like autonomous car-following model with deep reinforcement learning. *Transportation Research Part C: Emerging Technologies*, page 348–368, Dec 2018.
- [12] Ahmad El Sallab, Mohammed Abdou, Etienne Perot, and Senthil Yogamani. End-to-end deep reinforcement learning for lane keeping assist. *arXiv: Machine Learning, arXiv: Machine Learning*, Oct 2016.
- [13] Dong Li, Dan Zhao, Qichao Zhang, and Yaran Chen. Reinforcement learning and deep learning based lateral control for autonomous driving. *Cornell University - arXiv, Cornell University - arXiv*, Oct 2018.
- [14] Jingbo Sun, Xing Fang, and Qichao Zhang. Reinforcement learning driving strategy based on auxiliary task for multi-scenarios autonomous driving. In *2023 IEEE 12th Data Driven Control and Learning Systems Conference (DDCLS)*, May 2023.
- [15] Youtian Guo, Qichao Zhang, Junjie Wang, and Shasha Liu. Hierarchical reinforcement learning-based policy switching towards multi-scenarios autonomous driving. In *2021 International Joint Conference on Neural Networks (IJCNN)*, Jul 2021.
- [16] Jingke Wang, Yue Wang, Dongkun Zhang, Yezhou Yang, and Rong Xiong. Learning hierarchical behavior and motion planning for autonomous driving. In *2020 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, Oct 2020.
- [17] Yihan Hu, Jiazhi Yang, Li Chen, Keyu Li, Chonghao Sima, Xizhou Zhu, Siqui Chai, Senyao Du, Tianwei Lin, Wenhai Wang, Lewei Lu, Xiaosong Jia, Qiang Liu, Jifeng Dai, Yu Qiao, and Hongyang Li. Planning-oriented autonomous driving. Dec 2022.
- [18] Bo Jiang, Shaoyu Chen, Qing Xu, Bencheng Liao, Jiajie Chen, Helong Zhou, Qian Zhang, Wenyu Liu, Chang Huang, and Xinggang Wang. Vad: Vectorized scene representation for efficient autonomous driving. Mar 2023.
- [19] Peide Cai, Hengli Wang, Yuxiang Sun, and Ming Liu. Dq-gat: Towards safe and efficient autonomous driving with deep q-learning and graph attention networks. *IEEE Transactions on Intelligent Transportation Systems*, page 21102–21112, Nov 2022.
- [20] Shaoshuai Shi, Li Jiang, Dengxin Dai, and Bernt Schiele. Mtr++: Multi-agent motion prediction with symmetric scene modeling and guided intention querying. Jun 2023.
- [21] Jiquan Ngiam, Benjamin Caine, Vijay Vasudevan, Zhengdong Zhang, Hao-Tien Lewis Chiang, Jeffrey Ling, Rebecca Roelofs, Alex Bewley, Chenxi Liu, Ashish Venugopal, David J. Weiss, Ben Sapp, Zhifeng Chen, and Jonathon Shlens. Scene transformer: A unified architecture for predicting multiple agent trajectories. *Cornell University - arXiv, Cornell University - arXiv*, Jun 2021.
- [22] Haochen Liu, Zhiyu Huang, Xiaoyu Mo, and Chen Lv. Augmenting reinforcement learning with transformer-based scene representation learning for decision-making of autonomous driving. Aug 2022.
- [23] Zhiyu Huang, Haochen Liu, Jingda Wu, Wenhui Huang, and Chen Lv. Learning interaction-aware motion prediction model for decision-making in autonomous driving. Feb 2023.
- [24] Haitong Ma, Yang Guan, Shengbo Eben Li, Xiangteng Zhang, Sifa Zheng, and Jianyu Chen. Feasible actor-critic: Constrained reinforcement learning for ensuring statewide safety. *Cornell University - arXiv, Cornell University - arXiv*, May 2021.
- [25] Linrui Zhang, Qin Zhang, Li Shen, Bo Yuan, and Xueqian Wang. Saferl-kit: Evaluating efficient reinforcement learning methods for safe autonomous driving. Jun 2022.
- [26] Qisong Yang, Thiago D. Simão, Simon H Tindemans, and Matthijs T. J. Spaan. Wcsac: Worst-case soft actor critic for safety-constrained reinforcement learning. *Proceedings of the AAAI Conference on Artificial Intelligence*, page 10639–10646, Sep 2022.
- [27] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Neural Information Processing Systems, Neural Information Processing Systems*, Jun 2017.
- [28] Jun-Young Chung, Caglar Gulcehre, Kyunghyun Cho, and Yoshua Bengio. Empirical evaluation of gated recurrent neural networks on sequence modeling. *arXiv: Neural and Evolutionary Computing, arXiv: Neural and Evolutionary Computing*, Dec 2014.
- [29] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv: Learning, arXiv: Learning*, Jul 2017.
- [30] Bo Han, Quanming Yao, Xingrui Yu, Gang Niu, Miao Xu, Weihua Hu, Ivor W. Tsang, and Masashi Sugiyama. Co-teaching: Robust training of deep neural networks with extremely noisy labels. *Neural Information Processing Systems, Neural Information Processing Systems*, Jan 2018.
- [31] Ming Zhou, Jun Luo, Julian Vilella, Yaodong Yang, David Rusu, Jiayu Miao, Weinan Zhang, and et al. Smarts: Scalable multi-agent reinforcement learning training school for autonomous driving. *Cornell University - arXiv, Cornell University - arXiv*, Oct 2020.